

Reviewer Pack — Tutte Polynomial Rank Invariant Bounds Resolution Width

Entry f6296d995ea7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 07:56:52 UTC

Tutte Polynomial Rank Invariant Bounds Resolution Width

Entry ID: f6296d995ea7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 07:56:52 UTC

1. Conjecture

Field A (mathematical branch): Graph Polynomials **Field B** (complexity object): Resolution Width

Statement:

For a Tseitin formula derived from a graph G , the resolution width is at least $\Omega(\log(T(G; 1, 1)))$, where $T(G; x, y)$ is the Tutte polynomial evaluated at $(x, y) = (1, 1)$ (number of spanning trees of G).

Rationale (proposer's reasoning):

The Tutte polynomial's spanning tree count captures graph connectivity, which directly influences the expansion properties of Tseitin formulas. Resolution width depends on the graph's expansion, and spanning trees provide a combinatorial measure of this structure, offering a novel bridge between algebraic graph theory and proof complexity.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 919c252718f3ffe9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	SAFE	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: INCONCLUSIVE against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Tutte polynomial resolution width Tseitin formula - Tutte polynomial (1,1) resolution width bounds - spanning trees resolution width Tseitin formula

Top relevant hits considered: - [http://arxiv.org/abs/1708.08132v1] Topological Tutte Polynomial - [http://arxiv.org/abs/1906.02781v2] Tutte Polynomial Activities - [http://arxiv.org/abs/1705.10753v2] Tutte Polynomials of Symmetric Hyperplane Arrangements - [http://arxiv.org/abs/1012.2478v2] Packing spanning trees and the k -tree protocol - [http://arxiv.org/abs/0904.3087v4] Distributed Maintenance of Anytime Available Spanning Trees in Dynamic Networks - [http://arxiv.org/abs/1305.2661v2] On Improved Bounds on Bounded Degree Spanning Trees for Points in Arbitrary Dimension - [s2:2407.17947] Supercritical Size-Width Tree-Like Resolution Trade-Offs for Graph Isomorphism - [s2:10.1145/3378667] Cops-Robber Games and the Resolution of Tseitin Formulas

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_random_3_regular_graph(n):
    if n % 2 != 0 or n < 4:
        raise ValueError("n must be even and at least 4")

    G = [[] for _ in range(n)]
    edges = set()

    def add_edge(u, v):
        if (u, v) not in edges and (v, u) not in edges:
            G[u].append(v)
            G[v].append(u)
            edges.add((u, v))
            edges.add((v, u))

    for i in range(n):
        for j in range(i + 1, n):
            if len(G[i]) < 3 and len(G[j]) < 3:
                add_edge(i, j)

    return G

def tutte_polynomial(G, x, y):
    def dfs(v, parent):
        nonlocal count
        visited[v] = True
        for u in G[v]:
            if not visited[u]:
                dfs(u, v)
        if parent is None:
            count += 1
```

```

n = len(G)
visited = [False] * n
count = 0
dfs(0, None)
return Fraction(count)

def resolution_width(G):
    def dpll():
        assignment = {}

        def unit_propagate():
            for lit in clauses:
                if all(lit in assignment or -lit in assignment for lit in clauses):
                    continue
            value = None
            for lit in lit:
                if lit not in assignment and -lit not in assignment:
                    value = lit
                    break
            if value is None:
                return False
            assignment[value] = True
            return True

        def backtrack():
            while len(assignment) > 0:
                var = list(assignment.keys())[-1]
                del assignment[var]
                if unit_propagate():
                    return True
            return False

        return backtrack()

    clauses = []
    for u in range(len(G)):
        for v in G[u]:
            clauses.append([u + 1, -v - 1])
            clauses.append([-u - 1, v + 1])

    return len(clauses)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    G = generate_random_3_regular_graph(n)
    T_val = tutte_polynomial(G, 1, 1)

    if T_val <= 0:
        return {
            "metric_name": "resolution_width",
            "metric_value": None,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "T(G; 1, 1) is non-positive"
        }

```

```

    }

    resolution_width_val = resolution_width(G)

    return {
        "metric_name": "resolution_width",
        "metric_value": resolution_width_val,
        "instances_tested": 1,
        "conjecture_holds": resolution_width_val >= math.log(T_val),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results if r["metric_value"] is not None) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"resolution_width < c * log(T(G; 1, 1))\" first={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

TRIAL: {'metric_name': 'resolution_width', 'metric_value': 120, 'instances_tested': 1, 'conjecture_holds': True}
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76387aa.py", line 122, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76387aa.py", line 94, in run_trial
    G = generate_random_3_regular_graph(n)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c76387aa.py", line 20, in generate_random_3_regular_graph
    raise ValueError("n must be even and at least 4")
ValueError: n must be even and at least 4

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before collecting data; pre-registered support condition cannot be evaluated
| next: Fix graph generation parameters to handle even $n \geq 4$ and re-run with multiple seeds

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	138864
2	propose	ollama_remote	qwen3:8b	0	0	126445
3	preregistration	ollama_remote	qwen3:8b	0	0	31770
4	novelty	ollama_remote	qwen3:8b	0	0	26496
5	novelty	ollama_remote	qwen3:8b	0	0	32623
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45957
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12219
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19987
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12137
10	judge	ollama_remote	qwen3:8b	0	0	20700

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 467197 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/f6296d995ea7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f6296d995ea7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f6296d995ea7.tar.gz` (if generated)